

Comparison of BFS, DFS, and A* Search Algorithms for Solving a 2x2x2 Rubik's Cube

Introduction to AI & Ethics Project 1 Proposal

Alec Pike

apike5@charlotte.edu

University of North Carolina at Charlotte

Charlotte, North Carolina, USA

ABSTRACT

This proposal outlines a comparative empirical study of classical graph search algorithms—Breadth-First Search (BFS), Depth-First Search (DFS), and A* Search—applied to solving the $2 \times 2 \times 2$ Rubik's Cube (Pocket Cube). The project will implement a lightweight, pure Python solver framework to evaluate each algorithm in terms of solution optimality, nodes expanded, memory footprint, and execution runtime across varying scramble depths.

KEYWORDS

state-space search, planning, breadth-first search, depth-first search, A* search, heuristics, combination puzzle

1. Introduction & Problem

Definition

1.1 Problem Statement

The $2 \times 2 \times 2$ Rubik's Cube (Pocket Cube) is a 3-dimensional classic combinatorial puzzle consisting of 8 corner pieces and 24 visible sticker faces. The goal is to restore a scrambled cube configuration to a uniform color distribution on all six faces through a sequence of face rotations.

1.2 Domain Characteristics & State Space Complexity

- **State Space Size:** The total number of reachable physical configurations for a $2 \times 2 \times 2$ cube is:
$$\frac{8! \times 3^8}{24} = 7! \times 3^7 = 3,674,160 \text{ states}$$
- **God's Number:** The maximum optimal solution length from any valid configuration

is 11 moves under the Half-Turn Metric (HTM) or 14 moves under the Quarter-Turn Metric (QTM).

- **Challenge:** Despite the relatively small state space compared to the $3 \times 3 \times 3$ cube (4.3×10^{19} states [1]), uninformed search algorithms quickly suffer from exponential time and memory complexity as depth increases, highlighting the need for informed planning techniques.

2. Problem Formulation & Planning Context

2.1 State Representation

To efficiently represent and transition cube configurations in Python:

- **Primary Representation (Vector/Tuple Model):** A 24-element tuple of integers representing the 24 sticker facelets (4 facelets $\times$ 6 faces), indexed from 0 to 23.
 - *Example:* (0,0,0,0, 1,1,1,1, 2,2,2,2, 3,3,3,3, 4,4,4,4, 5,5,5,5) represents the solved cube state.
- **String Representation (Readability/Compatibility Model):** A 24-character string consisting of only the letters "W", "G", "R", "B", "O", and "Y", corresponding to the colors of the six faces, utilized by [1]. Equivalent and easily translatable to the tuple representation above.
 - *Example:* "WWWG GGGRRRRBBBBB OOOOYY" represents the solved cube state.
- **Efficiency & Hashability:** Immutable Python data structures (tuple) are required

to enable $O(1)$ lookup times in `set` and `dict` hash tables for closed/visited lists.

2.2 Initial and Goal Conditions

- **Initial State (S_0):** A valid scrambled state vector generated by applying k random legal face turns to the solved state, or imported from standard benchmark scrambles such as those computed by [1].
- **Goal Condition (S_{goal}):**
 - *Strict Goal:* The canonical state tuple where indices $[0..3]$ share color 0, $[4..7]$ share color 1, etc.
 - *Goal Test Function:* `is_goal(state)` returns `True` if each of the 6 faces contains uniform colors.

2.3 Available Actions (Transition Model)

- **Spatial Symmetry Reduction:** To prevent redundant rotations of the entire cube in 3D space, one corner piece (e.g., Bottom-Back-Left) is held fixed.
- **Action Space (A):** 6 legal primitive face rotations under the Quarter-Turn Metric (QTM):
 - U, U' (Up face clockwise 90° , counter-clockwise 90°)
 - R, R' (Right face clockwise 90° , counter-clockwise 90°)
 - F, F' (Front face clockwise 90° , counter-clockwise 90°)

The 180-degree turns $U2 R2 F2$ are not needed to solve the problem (they can be represented as sequences of two quarter-turns).

- **Transition Function $T(S, a) \rightarrow S'$:** A deterministic mapping in Python that permutes state vector indices according to pre-defined mapping arrays for each move $a \in A$.

2.4 Inclusion of Planning Principles

This problem belongs to **classical AI deterministic planning**:

- **Fully Observable:** The entire state of the cube is known at all times.
- **Deterministic:** Every action a yields a single deterministic successor state S' .
- **Static & Discrete:** No dynamic exogenous

changes; discrete action space and state transitions.

- **Path-Cost Planning:** The objective is to compute a sequence of macro-actions $\langle a_1, a_2, \dots, a_n \rangle$ from S_0 to S_{goal} while minimizing total step cost $C = \sum c(a_i)$ (where $c(a_i) = 1$).

3. Algorithms & Methodology

3.1 Algorithms to Implement

1. Breadth-First Search (BFS):

- *Implementation:* Queue-based (`collections.deque`) frontier with a visited set.
- *Role:* Serves as the baseline for finding guaranteed optimal paths in unweighted graphs. High space complexity $O(b^d)$.

2. Depth-First Search (DFS)/Iterative Deepening DFS (IDDFS):

- *Implementation:* Stack-based recursive search. Standard DFS will include depth bounds or use IDDFS to prevent infinite loops in cyclic graph spaces.
- *Role:* Evaluates memory efficiency $O(b \cdot d)$ at the potential cost of non-optimal path discovery.

3. A* Search:

- *Implementation:* Priority queue (heapq) ordering nodes by evaluation function $f(n) = g(n) + h(n)$, where $g(n)$ is path depth and $h(n)$ is an admissible heuristic.
- *Role:* Evaluates how heuristic information prunes the search tree to achieve optimal solutions with significantly fewer expanded nodes.

3.2 Draft Heuristic Ideas for A* Search

To guarantee that A* finds optimal solutions, heuristics must be admissible ($h(n) \leq h^*(n)$) and ideally also consistent.

- **Draft Heuristic 1: Misplaced Facelets (Hamming Distance) Bound**

$$h_1(n) = \left\lceil \frac{\text{no. of misplaced facelets}}{8} \right\rceil$$

- *Admissibility:* Each single face turn

changes exactly 8 sticker facelet positions (4 on the rotated face, 4 on adjacent layers). Thus, dividing misplaced stickers by 8 never overestimates the remaining move count.

- **Draft Heuristic 2: 3D Corner Manhattan/3D Position Distance**

$$h_2(n) = \left\lceil \frac{1}{4} \sum_{i=1}^7 \text{MDist}(\text{Corner}_i^{\text{current}}, \text{Corner}_i^{\text{target}}) \right\rceil$$

- *Admissibility:* Computes spatial 3D

Manhattan distances of corner positions from their target origins divided by 4 (since one move changes the position of exactly 4 corners).

- **Draft Heuristic 3: Corner Pattern Database (Abstraction Heuristic)**
 - Precomputes exact move distances for a simplified/abstracted model (e.g., tracking orientation/position of a 4-corner subset) stored in a Python dictionary.

4. Proposed Implementation Architecture & Python Design

```
rubiks_2x2_solver/
├── cube.py           # State representation, goal test, move permutation arrays
├── algorithms/
│   ├── bfs.py       # Breadth-First Search implementation
│   ├── dfs.py       # DFS / Iterative Deepening DFS implementation
│   └── astar.py      # A* Search implementation with priority queue
├── heuristics.py    # Misplaced facelets, 3D distance, and pattern database
└── heuristics
    └── benchmarks.ipynb # Test suite runner, performance logging, visualizations
```

5. Evaluation Plan & Metrics

Algorithms will be benchmarked on generated scramble sets categorized by scramble depth ($k = 1, 2, \dots, 11$).

Table 1: Summary of Evaluation Metrics

Evaluation Metric	Description
Solution Path Length	Number of moves in the generated solution sequence (optimality check).
Nodes Expanded	Total number of states removed from the frontier and evaluated.
Peak Memory Usage	Maximum memory allocated by frontier and visited data structures (in number of state nodes).
Execution Time	CPU clock time required to terminate (seconds).

Acknowledgements

This paper was drafted using Google Gemini 3.6 Thinking, based on an original idea.

References

1. Tim. 2016. Generating the Full List of Valid States of a 2×2 Rubik’s Cube. Retrieved from <http://tim.id.au/limejuice/generating-the-full-list-of-valid-states-of-a-2x2-rubiks-cube/>.